



Efficient, portabLe And Secure orchesTration for reliable servICes

TrustMee: Self-Verifying Remote Attestation Evidence

2026-05-05

Parsa Sadri Sinaki¹, Zainab Ahmad²¹, Wentao Xie²¹,
Merlijn Sebrechts³, Jimmy Kjällman², Lachlan J. Gunn¹

¹Aalto University ²Ericsson Finland ³Ghent University + imec



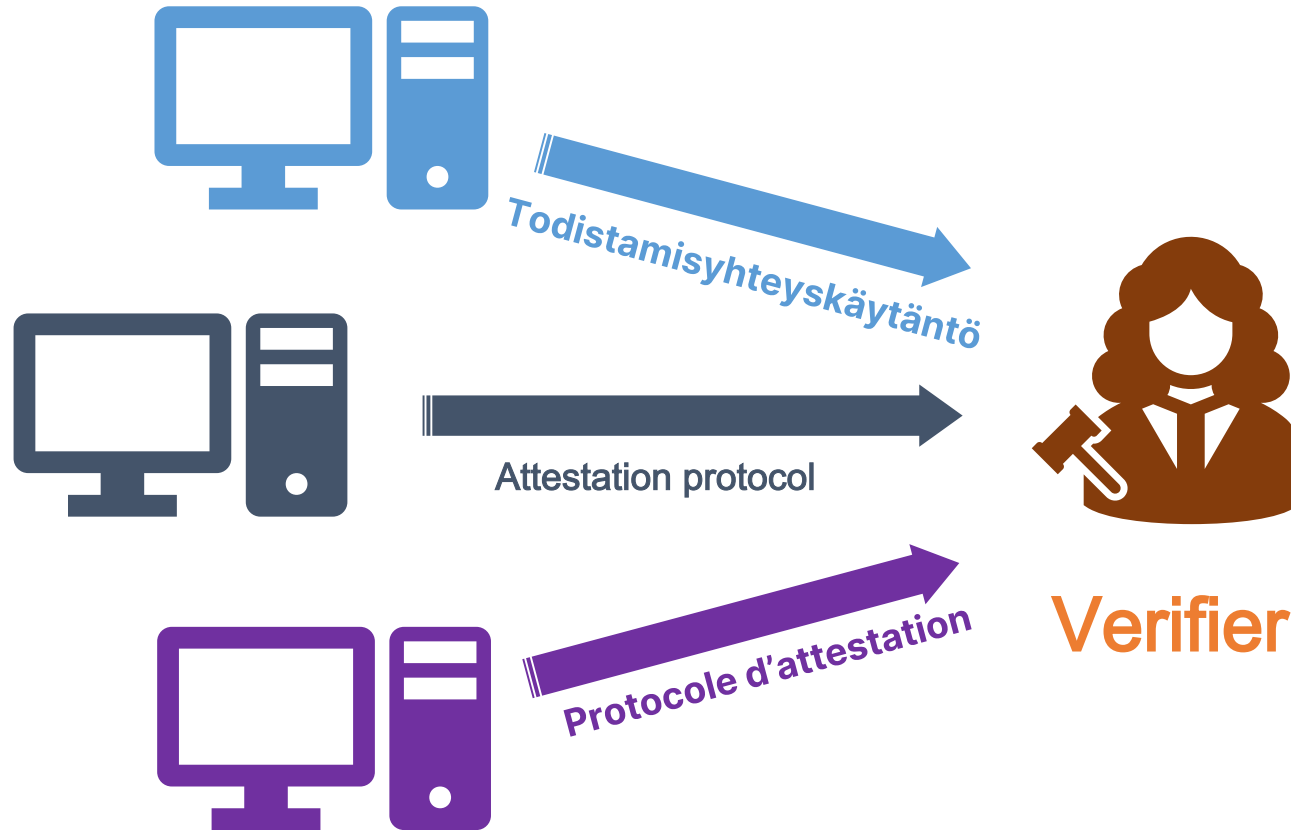
Co-funded by
The European Union



Funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or SNS-JU. Neither the European Union nor the granting authority can be held responsible for them.

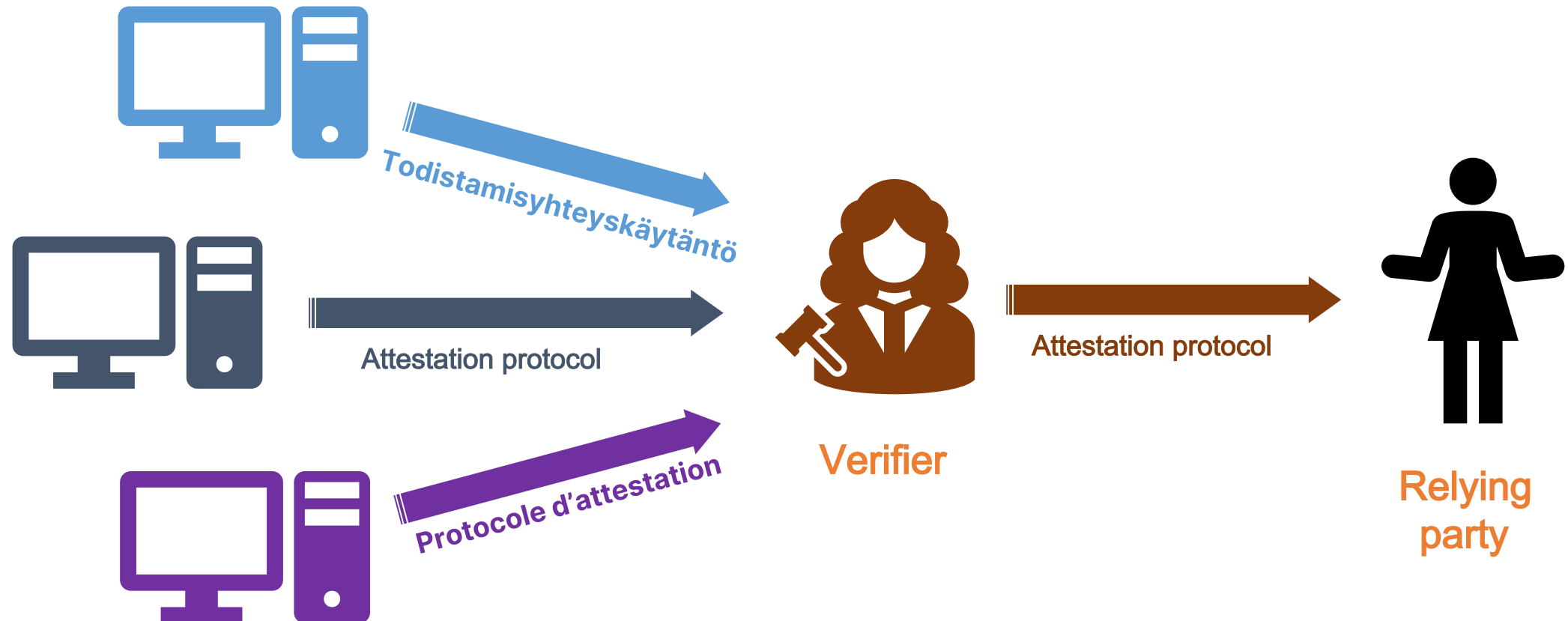
The problem

Attester
HW = Samsung A52
OS = Android 11
App = Bank ID



I'm talking to...
HW = Samsung A52
OS = Android 11
App = Bank ID

Verification frameworks



Solution: TrustMee

Idea: Attester provides verification code to verifier

- Eliminates all platform-specific code from the verifier

Problem: Malicious verification code can compromise the verifier

Solution: Protect the verifier using sandboxing techniques from the web

Problem: How can we trust the verification logic

Solution: Measure the verification code, now it's just another claim to check

WebAssembly: An application virtual machine

Applications compiled into WebAssembly modules

Modules have (non-exhaustive)

- An isolated memory space
- Exports: functionality exposed to the VMM (e.g. program entry point)
- Imports: functionality requested from the VMM (e.g. storage operations)

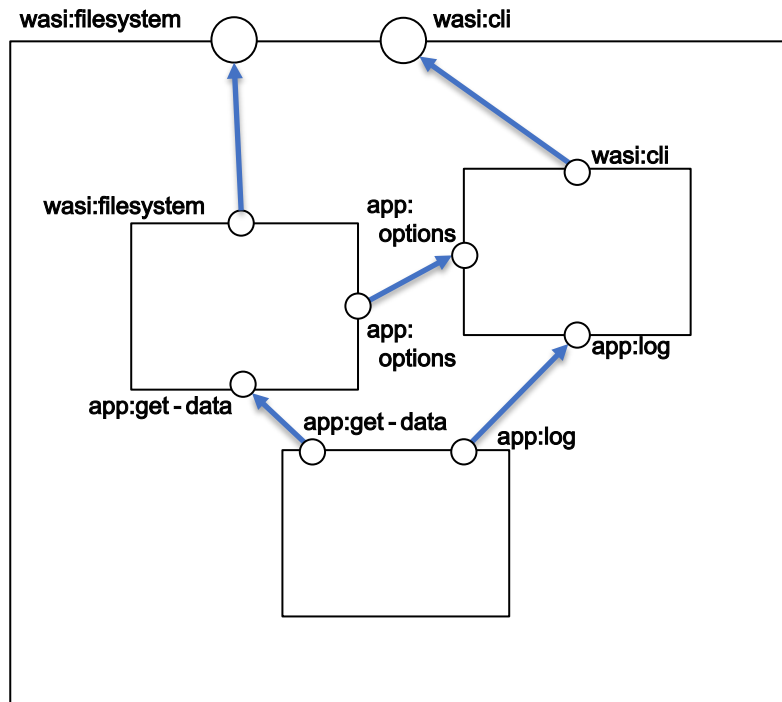
Wasm VMs are cheap!

- We can put different parts of an application each in their own sandbox

WebAssembly component model

Wasm modules can be wrapped into Components with a well-defined ABI

- Wasm components can be composed into larger components



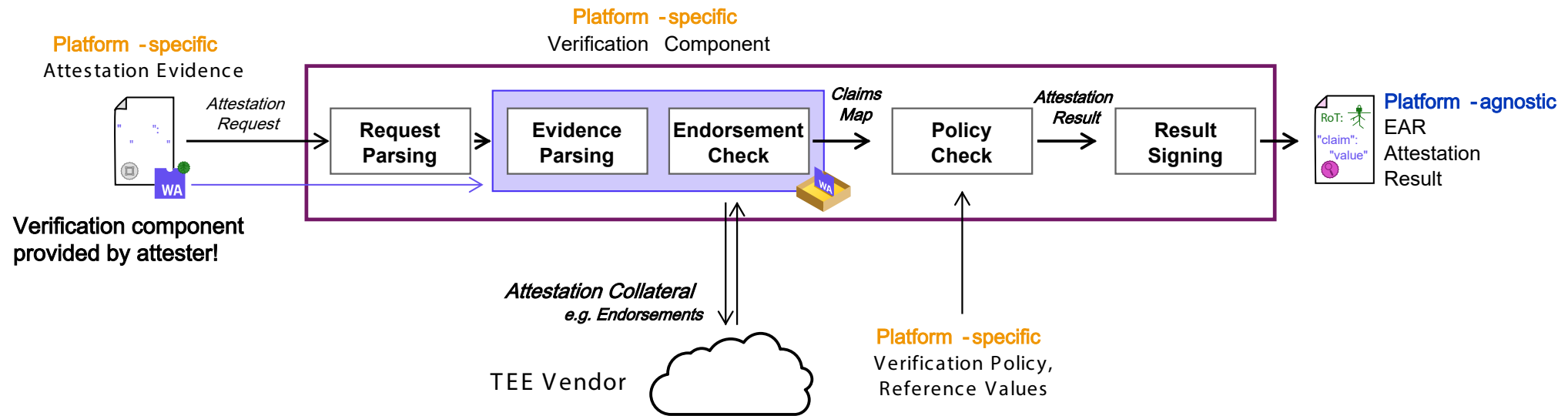
```
package trustee:verifier;

interface verifier-interface {
    variant optional-data {
        value(list<u8>),
        not-provided,
    }

    type report-data = optional-data;
    type init-data-hash = optional-data;

    resource verifier {
        constructor();
        evaluate: func(
            evidence: list<u8>,
            expected-report-data: report-data,
            expected-init-data-hash: init-data-hash,
        ) -> string;
    }
}

world verifier {
    export verifier-interface;
}
```



TrustMee implementation

Implemented as platform driver in Trustee

- Instantiates verification component and uses it to validate the evidence
- **CMW** wrapper for platform-specific evidence (CBOR or JSON)
- EAT output with platform-specific output in nested claims map

```
{
  "__cmwc_t": "https://trustmee.invalid/cmwc/verification - input",
  "evidence": [
    "application/eat - ucs+json; eat_profile= \"https://trustmee.invalid/eat/component - evidence\" ",
    "<base64url(EAT JSON payload)>",
    4
  ],
  "verifier": [
    "application/wasm",
    "<base64url(wasm bytes)>",
    2
  ],
  "snp-collateral": [
    "application/vnd.trustmee.snp - collateral+cbor",
    "<base64url(CBOR collateral)>",
    2
  ]
}
```

Input (CMW)

```
{
  "eat_profile" : "https://trustmee.invalid/eat/verification - result" ,
  "tee_type": "snp",
  "claims" : {
    "measurement": "<verifier - specific value>" ,
    "reported_tcb_snp" : 23
  },
  "verifier_component_sha256" : "<64 lowercase hex characters>" ,
  "verifier_component_signature_public_key" : "<public key>",
  "init_data" : "<optional>" ,
  "report_data" : "<optional>"
}
```

Output (EAT)

Verification component

Verification component in a Wasm sandbox

- Isolated filesystem region for caching, etc.
- Attester can send component, or verifier can download from OCI repository

Problem: Verification components isolated, but not completely safe

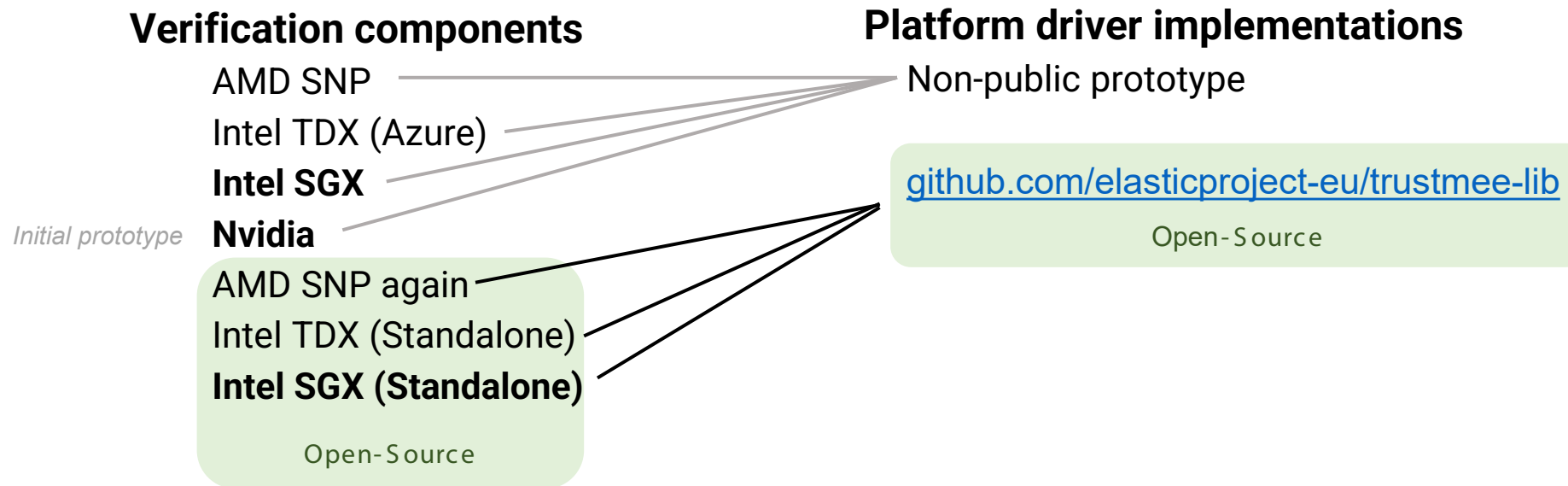
- Rogue components could cause DoS (infinite loops, etc.)
- Components needing network access (e.g. for OCSP) potentially risky

Solution: Additional restrictions on untrusted components

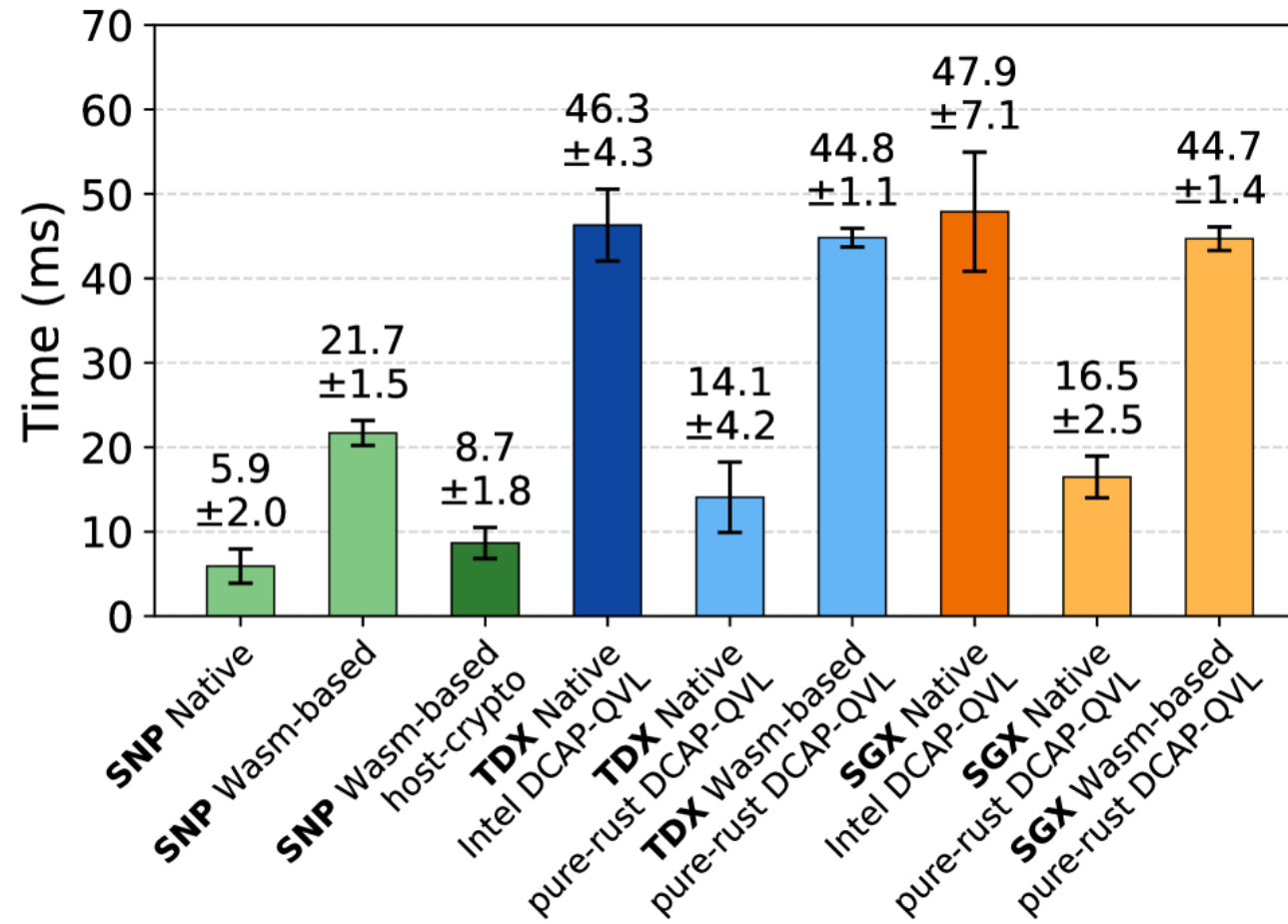
- Limited computation time with Wasmtime's "fuel" feature
- No network access for uncertified components
 - Collateral can be stapled

TrustMee implementation

Two implementations, one public



TrustMee implementation



Appraisal policies

Problem: Extra layer of verification requires additional policy checks

- Claims can't be trusted unless verification component also checked
- Hardware-specific claims nested in TrustMee EAT invalidates old policies

Solution: Automatically transform policies to check nested output

- Rewrite old checks to point to nested claims
- Add new reference values for component hash or signer

```
allow if {  
  input.snp.measurement == query_reference_value("snp_measurement")  
  input.snp.reported_tcb_snp >= query_reference_value("snp_min_tcb")  
}
```



```
allow if {  
  input.tee_type == "snp"  
  input.verifier_component_sha256 in query_reference_value("trusted_snp_components")  
  input.claims.measurement == query_reference_value("snp_measurement")  
  input.claims.reported_tcb_snp >= query_reference_value("snp_min_tcb")  
}
```

Attester modifications

Problem: Existing verification libraries don't produce TrustMee-compatible output

Solution: Attester-side library to wrap the evidence in our CMW format

- API still under development, not shown here
- Gets evidence from CoCo attestation agent (or provide your own)
- <https://github.com/elasticproject-eu/trustmee-attester>

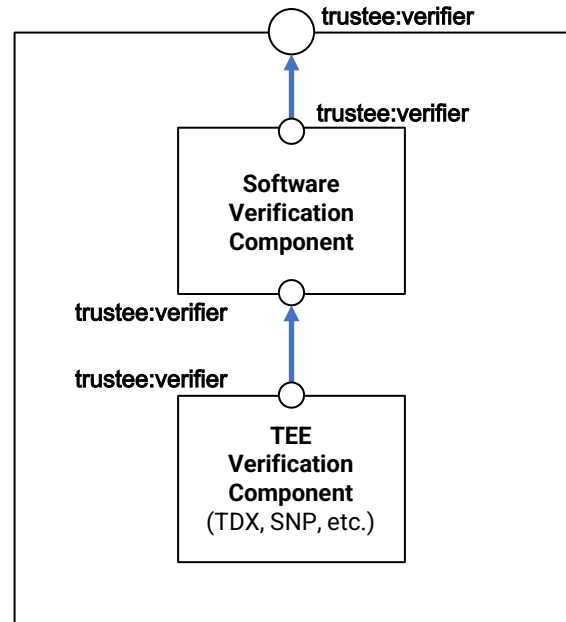
The Future



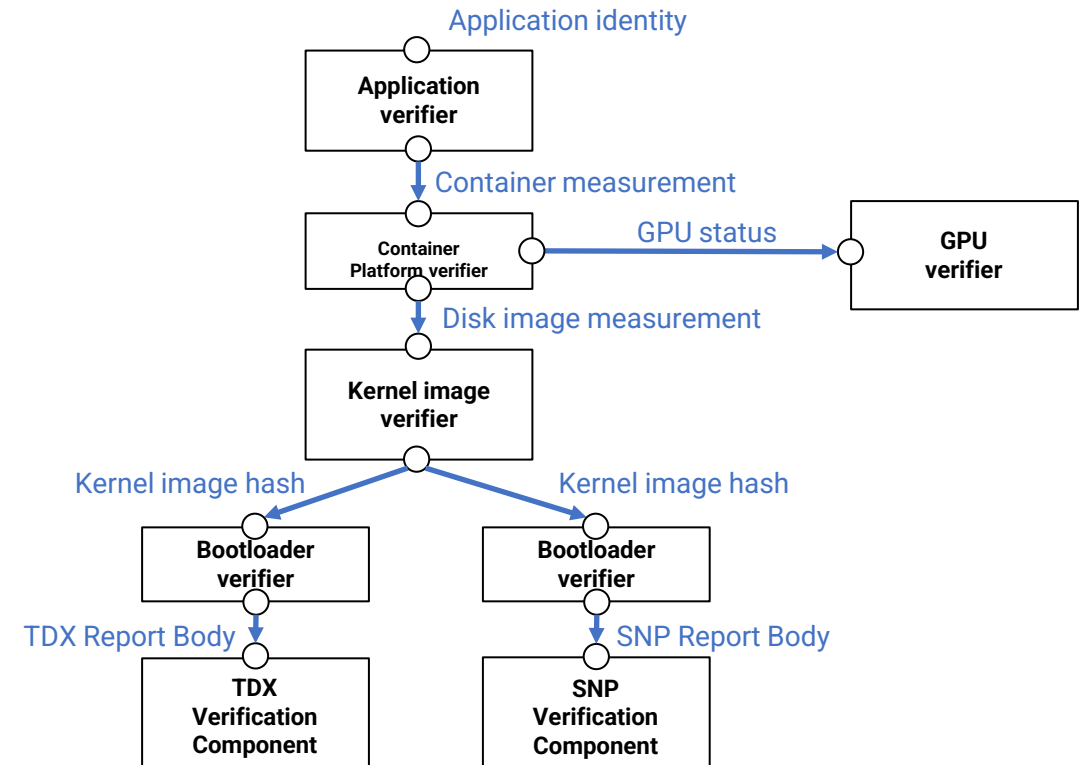
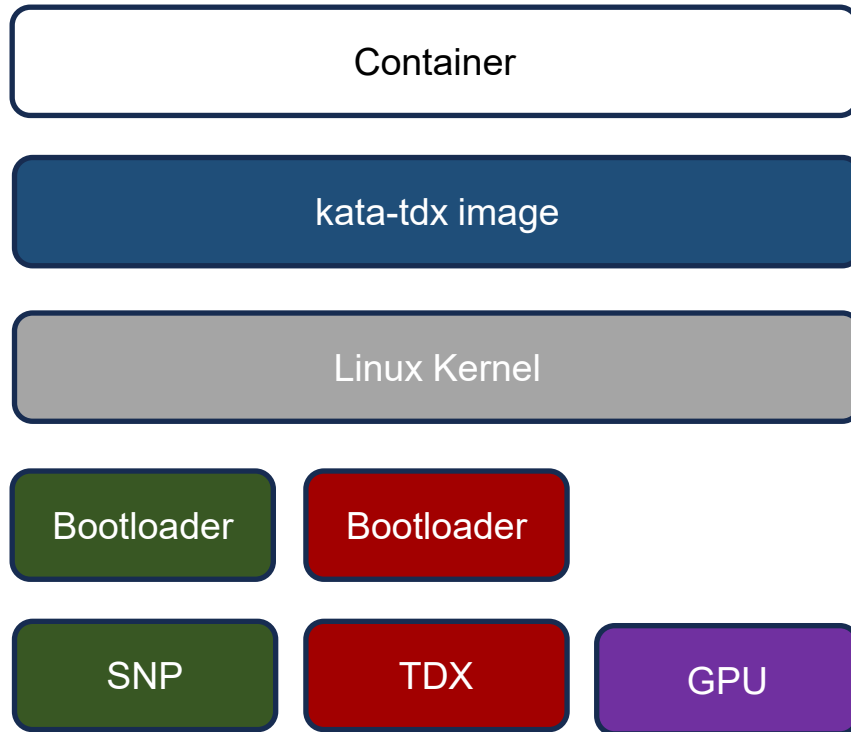
Layered & composite attestation

Problem: Attestation doesn't come just from hardware, but a whole software stack too

Solution: Build TrustMee verification components for higher-level application platforms

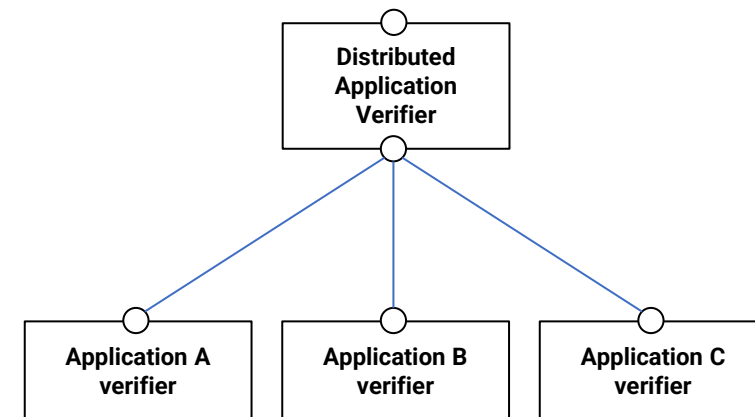
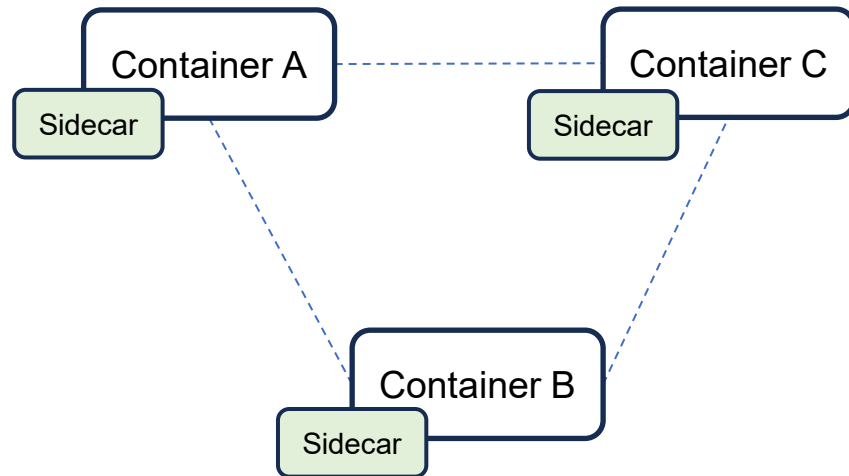


Example: container platform



Example: distributed applications

Consensus-like protocol combines component attestations into whole-app attestations





Immediate questions

How should we encapsulate the evidence?

- TrustMee evidence as an EAT token
 - TrustMee-specific EAT profile wrapping platform-specific evidence
 - Platform-specific EAT profile, TrustMee metadata in JOSE wrapper
 - TrustMee = signature algorithm
 - evidence = signature
 - verification component = public key
 - Allows any EAT profile to self-authenticate without transformation by vendor's attestation service
- TrustMee evidence only as a CMW collection type, don't try to force non-EAT evidence into EAT
 - CMW needed anyway for stapled endorsements

**Current
implementation**

How should we encapsulate the results (as a submodn the EAR document)?

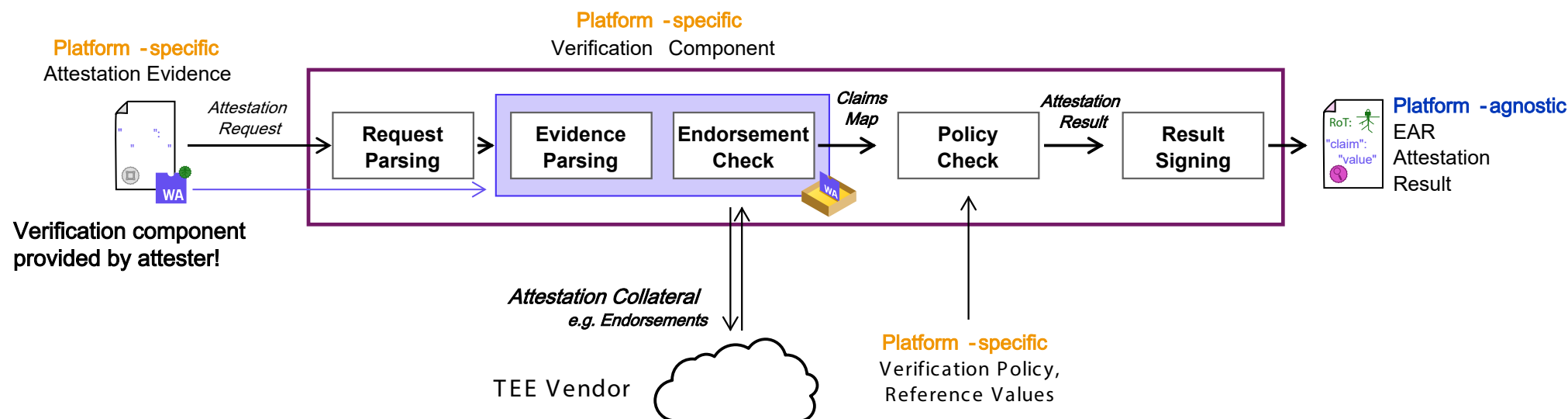
- TrustMee as lead attester, with the verification component producing a nested token
 - Inserts claims list as extra layer
 - No profile-compliance issues, since the nested token doesn't have extra claims not in its profile
- TrustMee claims inserted into existing output
 - Resulting output doesn't comply with any EAT profile

**Current
implementation**

Reference Values

What's the best way to deal with reference values for composite attesters?

- Not currently addressed in TrustMee
- Should verifier components be able to provide or validate reference values themselves?



Summary

TrustMee allows attestation verifiers to eliminate platform-dependent logic

Platform-specific logic can be shared between different verifiers

Prototype available as open-source

- Standalone verification library
- Trustee platform driver



arxiv.org/abs/2602.13148



[github.com/elasticproject - eu/trustmee - lib](https://github.com/elasticproject/eu-trustmee-lib)